

Jetpack Compose第五章第二节教学讲义

(Compose附带效应全解析)

各位同学，上节课我们解决了Compose与传统View的协同问题，今天要攻克Compose开发中的一个核心难点——**附带效应（Side Effect）**。很多同学刚接触Compose时会疑惑：为什么我在Compose函数里写网络请求会崩溃？为什么定时器会重复创建？其实这些问题都和“附带效应”有关。这节课我们就从“是什么、为什么、怎么用”三个维度，把Compose的效应API讲透，让你写出稳定、高效的Compose代码。

一、核心认知：先搞懂“附带效应”是什么

1. 从生活场景理解效应

我们先抛开技术，用生活场景类比：假设你是个厨师（Compose函数），核心工作是“做菜”（构建UI）。在做菜过程中，你可能需要“打开抽油烟机”“给食材称重”“关火”——这些不是“做菜”本身，但又是做菜必须做的操作，这些就是“附带效应”。

对应到Compose中：

- 核心工作（无副作用）**：根据状态（如“食材种类”“口味”）输出UI（如“宫保鸡丁”“鱼香肉丝”），这是Compose的本职，特点是“纯函数”——相同输入一定得到相同输出，不会影响外部环境；
- 附带效应（有副作用）**：在构建UI时，需要执行的“非UI构建操作”，比如网络请求、启动定时器、注册监听器、修改全局变量等，这些操作会影响外部环境，或依赖外部资源。

2. 为什么Compose要“特殊处理”效应？

因为Compose函数有个关键特性——**重组（Recomposition）**：当状态变化时，Compose会重新执行函数来更新UI。如果直接在Compose函数里写效应代码，会出现大问题：

Code block

```
1 // 错误示范：直接在Compose函数中写网络请求（附带效应）
2 @Composable
3 fun BadEffectDemo() {
4     val text = remember { mutableStateOf("初始文本") }
5
6     // 错误：重组时会重复执行，导致多次网络请求
7     val result = api.getSomeData() // 网络请求（附带效应）
8     text.value = result.data
```

```
9
10     Text(text = text.value)
11 }
12
```



问题总结：① 重组触发多次执行（如网络请求发十几次）；② 生命周期不匹配（如页面销毁后定时器还在跑，导致内存泄漏）；③ 线程问题（如在UI线程做耗时操作导致卡顿）。

3. 核心解决思路

Compose提供了一系列“效应API”，本质是帮我们实现：**控制效应的执行时机、执行次数，以及与Compose组件的生命周期绑定**。简单说就是：让效应“该执行时执行，该停止时停止”。

二、基础效应API：LaunchedEffect——最常用的“启动效应”

当你需要在Compose组件“出现时”启动一个协程（比如网络请求、定时器），并且在组件“消失时”自动取消协程，`LaunchedEffect` 就是你的首选。它是Compose中最基础、最常用的效应API。

1. 核心作用

- 在Compose组件的“生命周期作用域”内启动协程；
- 组件重组时，只有“关键参数”变化才会重新启动协程；
- 组件被销毁（从组合树移除）时，自动取消协程，避免内存泄漏。

2. 实战1：用LaunchedEffect实现“延迟显示文本”

需求：组件加载后，延迟2秒显示“加载完成”文本，页面关闭时不会有残留操作。

Code block

```
1  import androidx.compose.foundation.layout.Arrangement
2  import androidx.compose.foundation.layout.Column
3  import androidx.compose.foundation.layout.fillMaxSize
4  import androidx.compose.material3.Text
5  import androidx.compose.runtime.*
6  import androidx.compose.ui.Alignment
7  import androidx.compose.ui.unit.sp
8  import kotlinx.coroutines.delay
```

```

9
10 @Composable
11 fun LaunchedEffectDemo() {
12     // 状态：控制显示的文本
13     var displayText by remember { mutableStateOf("加载中...") }
14
15     // 启动效应：在组件作用域内启动协程
16     LaunchedEffect(key1 = Unit) { // key1是关键参数，后面讲
17         // 这里的代码运行在协程中，不会阻塞UI线程
18         delay(2000) // 延迟2秒（模拟网络请求等耗时操作）
19         displayText = "加载完成！" // 更新状态，触发重组
20     }
21
22     // 构建UI
23     Column(
24         modifier = Modifier.fillMaxSize(),
25         horizontalAlignment = Alignment.CenterHorizontally,
26         verticalArrangement = Arrangement.Center
27     ) {
28         Text(text = displayText, fontSize = 24.sp)
29     }
30 }
31

```

3. 关键参数：key（控制效应是否重启）

`LaunchedEffect` 的第一个参数是 `key`（可以传多个，用 `key1, key2...`），它是控制效应“是否重新执行”的开关，核心规则：

- 组件首次组合时，效应一定会执行；
- 组件重组时，只有当 `key` 的值发生变化，效应才会“取消旧协程，启动新协程”；
- 如果 `key` 不变，重组时效应不会重新执行。

实战2：用key控制“搜索输入防抖”

需求：用户输入搜索关键词后，延迟500毫秒再执行搜索，输入变化时重新计时（防抖）。

Code block

```

1 import androidx.compose.foundation.layout.Column
2 import androidx.compose.foundation.layout.padding
3 import androidx.compose.material3.OutlinedTextField
4 import androidx.compose.material3.Text
5 import androidx.compose.runtime.*
6 import androidx.compose.ui.Modifier
7 import androidx.compose.ui.unit.dp

```

```

8  import kotlinx.coroutines.delay
9
10 @Composable
11 fun SearchDebounceDemo() {
12     // 状态：搜索输入框文本
13     var searchText by remember { mutableStateOf("") }
14     // 状态：显示搜索结果
15     var searchResult by remember { mutableStateOf("") }
16
17     // 启动效应：key为searchText，输入变化时重启协程
18     LaunchedEffect(key1 = searchText) {
19         if (searchText.isBlank()) {
20             searchResult = ""
21             return@LaunchedEffect
22         }
23         // 延迟500毫秒（防抖：用户停止输入500ms后再搜索）
24         delay(500)
25         // 模拟搜索接口调用
26         searchResult = "搜索 '$searchText' 的结果：找到10条数据"
27     }
28
29     Column(modifier = Modifier.padding(16.dp)) {
30         OutlinedTextField(
31             value = searchText,
32             onValueChange = { searchText = it },
33             label = { Text("请输入搜索关键词") },
34             modifier = Modifier.fillMaxWidth()
35         )
36         Text(
37             text = searchResult,
38             modifier = Modifier.padding(top = 16.dp)
39         )
40     }
41 }
42

```



效果说明：用户快速输入“安卓”时，每输入一个字都会触发重组，但由于 `searchText` 变化，旧协程会被取消，新协程启动并重新计时，只有当用户停止输入500毫秒后，才会执行真正的搜索，避免频繁调用接口。

三、作用域API：rememberCoroutineScope——获取“可手动控制”的协程作用域

LaunchedEffect 的协程是“自动管理”的，而 rememberCoroutineScope 会给你一个“手动控制”的协程作用域，适合需要“按需启动/取消”的场景（比如按钮点击触发网络请求，点击取消按钮终止请求）。

1. 核心区别

API	协程启动时机	控制方式	适用场景
LaunchedEffect	组件组合时自动启动	通过key自动控制重启/取消	组件生命周期绑定 页面加载时请求数
rememberCoroutineScope	手动调用launch启动	通过Job手动控制取消	用户交互触发的操 点击请求，需手动

2. 实战3：手动控制网络请求的启动与取消

需求：点击“开始请求”按钮发起网络请求，点击“取消请求”按钮终止请求，请求过程中显示“加载中”。

Code block

```
1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.foundation.layout.padding
3  import androidx.compose.material3.Button
4  import androidx.compose.material3.Text
5  import androidx.compose.runtime.*
6  import androidx.compose.ui.Modifier
7  import androidx.compose.ui.unit.dp
8  import kotlinx.coroutines.CoroutineScope
9  import kotlinx.coroutines.delay
10 import kotlinx.coroutines.launch
11
12 @Composable
13 fun RememberCoroutineScopeDemo() {
14     // 1. 获取手动控制的协程作用域
15     val scope = rememberCoroutineScope()
16     // 2. 状态：请求状态和结果
17     var isLoading by remember { mutableStateOf(false) }
18     var requestResult by remember { mutableStateOf("") }
19     // 3. 用于控制协程的Job（可理解为协程的“身份证”）
```

```

20     var requestJob by remember { mutableStateOf<kotlinx.coroutines.Job?>(null)
21 }
22 Column(modifier = Modifier.padding(16.dp)) {
23     // 开始请求按钮
24     Button(
25         onClick = {
26             if (isLoading) return@Button // 防止重复点击
27             isLoading = true
28             requestResult = ""
29             // 手动启动协程，并用Job记录
30             requestJob = scope.launch {
31                 try {
32                     // 模拟网络请求（延迟3秒）
33                     delay(3000)
34                     requestResult = "请求成功！数据：{name: '安卓'}"
35                 } catch (e: Exception) {
36                     if (e.message == "Job was cancelled") {
37                         requestResult = "请求已取消"
38                     } else {
39                         requestResult = "请求失败：${e.message}"
40                     }
41                 } finally {
42                     isLoading = false
43                 }
44             }
45         },
46         enabled = !isLoading,
47         modifier = Modifier.padding(bottom = 8.dp)
48     ) {
49         Text(text = if (isLoading) "加载中..." else "开始请求")
50     }
51
52     // 取消请求按钮
53     Button(
54         onClick = {
55             // 手动取消协程
56             requestJob?.cancel()
57         },
58         enabled = isLoading,
59         modifier = Modifier.padding(bottom = 16.dp)
60     ) {
61         Text(text = "取消请求")
62     }
63
64     // 显示结果
65     Text(text = requestResult)

```

```
66     }
67 }
68
```



关键知识点：① `rememberCoroutineScope` 的作用域与组件生命周期绑定，组件销毁时作用域内的协程会自动取消；② `Job` 是协程的句柄，通过 `cancel()` 可手动终止协程。

四、状态保鲜API：rememberUpdateState——“记住”最新状态

当效应中引用了某个状态，但又不想因为这个状态变化而重启效应时，`rememberUpdateState` 就派上用场了。它能让效应“始终使用最新的状态值”，同时不触发效应重启。

1. 问题场景

假设用 `LaunchedEffect` 启动一个3秒的定时器，期间如果“目标文本”状态变化，我们希望定时器结束后使用最新的文本，但又不想重启定时器：

Code block

```
1  // 问题示范：状态变化会导致LaunchedEffect重启
2  @Composable
3  fun BadRememberDemo() {
4      var targetText by remember { mutableStateOf("初始目标") }
5      var resultText by remember { mutableStateOf("等待定时器...") }
6
7      // 问题：targetText变化时，LaunchedEffect会重启，定时器重新计时
8      LaunchedEffect(key1 = targetText) {
9          delay(3000)
10         resultText = "定时器结束，目标：$targetText"
11     }
12
13     Column(modifier = Modifier.padding(16.dp)) {
14         Button(onClick = { targetText = "更新后的目标" }) {
15             Text(text = "修改目标文本")
16         }
17         Text(text = resultText, modifier = Modifier.padding(top = 16.dp))
18     }
19 }
20
```

2. 实战4：用rememberUpdateState解决“状态保鲜”问题

Code block

```
1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.foundation.layout.padding
3  import androidx.compose.material3.Button
4  import androidx.compose.material3.Text
5  import androidx.compose.runtime.*
6  import androidx.compose.ui.Modifier
7  import androidx.compose.ui.unit.dp
8  import kotlinx.coroutines.delay
9
10 @Composable
11 fun RememberUpdateStateDemo() {
12     var targetText by remember { mutableStateOf("初始目标") }
13     var resultText by remember { mutableStateOf("等待定时器...") }
14
15     // 1. 用rememberUpdateState“保鲜”最新状态
16     val latestTargetText by rememberUpdateState(newValue = targetText)
17
18     // 2. key用Unit, 确保定时器只启动一次
19     LaunchedEffect(key1 = Unit) {
20         delay(3000)
21         // 3. 引用保鲜后的状态, 始终是最新值
22         resultText = "定时器结束, 目标: $latestTargetText"
23     }
24
25     Column(modifier = Modifier.padding(16.dp)) {
26         Button(
27             onClick = { targetText = "更新后的目标" },
28             modifier = Modifier.padding(bottom = 16.dp)
29         ) {
30             Text(text = "修改目标文本")
31         }
32         Text(text = resultText)
33     }
34 }
35
```



效果说明：点击“修改目标文本”后，定时器不会重启，但3秒后会显示“更新后的目标”。核心原理：`rememberUpdateState` 会在状态变化时更新内部值，但不会触发依赖

它的效应重启。

五、资源释放API: DisposableEffect——“用完记得关”的效应

当你需要“注册监听器”“初始化资源”等操作，并且必须在组件销毁或效应重启前“取消监听”“释放资源”时，`DisposableEffect` 是唯一选择。它的核心是“可释放”——有明确的“清理逻辑”。

1. 核心作用

- 执行需要“清理”的效应（如注册广播、传感器监听、生命周期监听）；
- 当 `key` 变化或组件销毁时，会先执行清理逻辑，再执行新的效应；
- 必须通过 `onDispose` 方法指定清理逻辑，否则编译报错（强制你释放资源）。

2. 实战5：监听Activity生命周期（用DisposableEffect）

需求：监听Activity的“resumed”和“paused”状态，在页面恢复时显示“页面活跃”，暂停时显示“页面后台”。

Code block

```
1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.foundation.layout.fillMaxSize
3  import androidx.compose.material3.Text
4  import androidx.compose.runtime.*
5  import androidx.compose.ui.Alignment
6  import androidx.compose.ui.Modifier
7  import androidx.compose.ui.unit.sp
8  import androidx.lifecycle.Lifecycle
9  import androidx.lifecycle.LifecycleEventObserver
10 import androidx.lifecycle.LocalLifecycleOwner
11
12 @Composable
13 fun DisposableEffectDemo() {
14     // 状态：显示生命周期状态
15     var lifecycleState by remember { mutableStateOf("初始状态") }
16     // 获取当前Activity的生命周期所有者
17     val lifecycleOwner = LocalLifecycleOwner.current
18
19     // 可释放效应：注册生命周期监听
20     DisposableEffect(key1 = lifecycleOwner) {
21         // 1. 创建生命周期观察者
```

```

22     val observer = LifecycleEventObserver { _, event ->
23         lifecycleState = when (event) {
24             Lifecycle.Event.ON_RESUME -> "页面活跃 (resumed) "
25             Lifecycle.Event.ON_PAUSE -> "页面后台 (paused) "
26             Lifecycle.Event.ON_STOP -> "页面停止 (stopped) "
27             Lifecycle.Event.ON_DESTROY -> "页面销毁 (destroyed) "
28             else -> lifecycleState
29         }
30     }
31
32     // 2. 注册观察者 (效应逻辑)
33     lifecycleOwner.lifecycle.addObserver(observer)
34
35     // 3. 必须指定清理逻辑: 取消注册
36     onDispose {
37         lifecycleOwner.lifecycle.removeObserver(observer)
38     }
39 }
40
41 // 显示状态
42 Column(
43     modifier = Modifier.fillMaxSize(),
44     horizontalAlignment = Alignment.CenterHorizontally,
45     verticalArrangement = Arrangement.Center
46 ) {
47     Text(text = "当前页面状态: ", fontSize = 20.sp)
48     Text(text = lifecycleState, fontSize = 24.sp, modifier =
Modifier.padding(top = 8.dp))
49 }
50 }
51

```



关键提醒：如果不通过 `onDispose` 取消注册，会导致生命周期所有者（Activity）被观察者持有，引发内存泄漏。`DisposableEffect` 通过强制要求写清理逻辑，帮我们避免这类问题。

六、轻量效应API：SideEffect——“每次重组都执行”的效应

如果你需要一个“在每次重组完成后都执行”的轻量级效应（比如同步状态到全局变量），`SideEffect` 是最佳选择。它没有key参数，每次重组都会执行，且执行在UI线程。

1. 适用场景

同步Compose状态到Compose外部的代码（如传统View的ViewModel、全局配置），且操作是轻量的（不耗时）。

2. 实战6：用SideEffect同步状态到全局变量

Code block

```
1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.foundation.layout.padding
3  import androidx.compose.material3.Switch
4  import androidx.compose.material3.Text
5  import androidx.compose.runtime.*
6  import androidx.compose.ui.Modifier
7  import androidx.compose.ui.unit.dp
8
9  // 全局配置类（Compose外部的代码）
10 object AppGlobalConfig {
11     var isDarkMode: Boolean = false
12     private set
13
14     // 供外部读取状态的方法
15     fun getDarkModeStatus(): String {
16         return "全局深色模式: ${if (isDarkMode) "开启" else "关闭"}"
17     }
18 }
19
20 @Composable
21 fun SideEffectDemo() {
22     // Compose内部状态：控制深色模式开关
23     var isDarkModeChecked by remember { mutableStateOf(false) }
24
25     // 轻量效应：每次重组都同步状态到全局变量
26     SideEffect {
27         AppGlobalConfig.isDarkMode = isDarkModeChecked
28         // 注意：这里不能做耗时操作，会阻塞UI线程
29     }
30
31     Column(modifier = Modifier.padding(16.dp)) {
32         // 深色模式开关
33         Column(horizontalAlignment = Alignment.CenterHorizontally) {
34             Switch(
35                 checked = isDarkModeChecked,
36                 onCheckedChange = { isDarkModeChecked = it }
37             )
38         }
39     }
40 }
```

```

38         Text(text = "Compose内部状态: ${if (isDarkModeChecked) "开启" else
    "关闭"}")
39     }
40
41     // 显示全局变量的状态 (模拟外部代码使用)
42     Text(
43         text = AppGlobalConfig.getDarkModeStatus(),
44         modifier = Modifier.padding(top = 16.dp)
45     )
46 }
47 }
48

```



注意事项：① `SideEffect` 每次重组都会执行，避免在这里写网络请求、定时器等重操作；② 执行在UI线程，耗时操作会导致卡顿。

七、状态生产API：produceState——“从外部数据源生产状态”

当你需要从“非Compose协程”的数据源（如Flow、回调、定时器）生产Compose状态时，`produceState` 可以帮你将外部数据“转换”为Compose可观察的状态，并且自动管理数据源的生命周期。

1. 核心优势

将外部数据源（如网络Flow、传感器数据）与Compose状态绑定，组件销毁时自动停止数据获取，无需手动管理。

2. 实战7：用produceState从Flow生产状态

需求：模拟一个“每秒递增”的Flow数据源，将其转换为Compose状态并显示，组件销毁时停止递增。

Code block

```

1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.foundation.layout.fillMaxSize
3  import androidx.compose.material3.Text
4  import androidx.compose.runtime.*
5  import androidx.compose.ui.Alignment
6  import androidx.compose.ui.Modifier

```

```

7  import androidx.compose.ui.unit.sp
8  import kotlinx.coroutines.delay
9  import kotlinx.coroutines.flow.flow
10
11 // 模拟外部数据源：每秒发送一个递增的数字
12 fun getIncrementFlow(): kotlinx.coroutines.flow.Flow<Int> {
13     return flow {
14         var count = 0
15         while (true) {
16             delay(1000)
17             emit(count++) // 发送数据
18         }
19     }
20 }
21
22 @Composable
23 fun ProduceStateDemo() {
24     // 从外部Flow生产Compose状态
25     val countState by produceState(
26         initialValue = 0, // 初始状态值
27         key1 = Unit // 控制是否重新生产状态的key
28     ) {
29         // 收集Flow数据，并更新状态
30         getIncrementFlow().collect { count ->
31             value = count // value是produceState的内置状态
32         }
33     }
34
35     // 显示状态
36     Column(
37         modifier = Modifier.fillMaxSize(),
38         horizontalAlignment = Alignment.CenterHorizontally,
39         verticalArrangement = Arrangement.Center
40     ) {
41         Text(text = "当前计数: ", fontSize = 20.sp)
42         Text(text = "$countState", fontSize = 32.sp, modifier =
43             Modifier.padding(top = 8.dp))
44     }
45 }

```



核心原理：① `produceState` 内部会启动协程收集数据源；② 组件销毁时，会自动取消协程，停止收集数据；③ `value` 是内置的可观察状态，更新后会触发重组。

八、派生状态API：derivedStateOf——“由其他状态计算而来”的状态

当一个状态是“由多个其他状态计算而来”，并且你希望“只有当依赖的状态变化时，才重新计算”，`derivedStateOf` 能帮你优化性能，避免不必要的计算。

1. 问题场景

如果直接在Compose函数中计算派生状态，每次重组都会重新计算，即使依赖的状态没变化：

Code block

```
1 // 问题示范：不必要的计算
2 @Composable
3 fun BadDerivedStateDemo() {
4     val a by remember { mutableStateOf(1) }
5     val b by remember { mutableStateOf(2) }
6     // 问题：每次重组都会重新计算a+b，即使a和b都没变化
7     val sum = a + b
8
9     Text(text = "a + b = $sum")
10 }
11
```

2. 实战8：用derivedStateOf优化“列表滚动状态”

需求：列表滚动时，当第一个可见项索引大于0时，显示“回到顶部”按钮，否则隐藏。这里“按钮显示状态”是由“滚动状态”派生而来的。

Code block

```
1 import androidx.compose.foundation.verticalScroll
2 import androidx.compose.material3.Button
3 import androidx.compose.material3.Text
4 import androidx.compose.runtime.*
5 import androidx.compose.ui.Modifier
6 import androidx.compose.ui.layout.ScrollPosition
7 import androidx.compose.ui.unit.dp
8 import androidx.compose.foundation.layout.*
9
10 @Composable
11 fun DerivedStateOfDemo() {
12     // 滚动状态：记录滚动位置
13     val scrollPosition = rememberScrollPosition()
```

```
14 // 原始状态: 第一个可见项的索引 (模拟列表滚动数据)
15 val firstVisibleItemIndex by remember { mutableStateOf(0) }
16
17 // 派生状态: 由firstVisibleItemIndex计算而来
18 val showToTopButton by remember {
19     derivedStateOf {
20         // 只有当firstVisibleItemIndex变化时, 才重新计算
21         firstVisibleItemIndex > 0
22     }
23 }
24
25 Column(modifier = Modifier.fillMaxSize()) {
26     // 回到顶部按钮 (根据派生状态显示/隐藏)
27     if (showToTopButton) {
28         Button(
29             onClick = { /* 回到顶部逻辑 */ },
30             modifier = Modifier
31                 .fillMaxWidth()
32                 .padding(8.dp)
33         ) {
34             Text(text = "回到顶部")
35         }
36     }
37
38     // 模拟长列表
39     Column(
40         modifier = Modifier
41             .weight(1f)
42             .verticalScroll(scrollPosition)
43             .padding(16.dp)
44     ) {
45         repeat(50) { index ->
46             Text(
47                 text = "列表项 $index",
48                 modifier = Modifier.padding(vertical = 8.dp),
49                 fontSize = 18.sp
50             )
51         }
52     }
53
54     // 模拟滚动操作 (改变firstVisibleItemIndex)
55     Button(
56         onClick = {
57             // 模拟滚动到第5项
58             firstVisibleItemIndex = 5
59         },
60         modifier = Modifier.padding(8.dp)
```

```

61         ) {
62             Text(text = "模拟滚动到第5项")
63         }
64     }
65 }
66

```



优化点： `derivedStateOf` 会缓存计算结果，只有当依赖的状态（`firstVisibleItemIndex`）变化时，才重新执行计算逻辑，避免每次重组都做无用功，提升性能。

九、状态流API： `snapshotFlow`——“将Compose状态转换为Flow”

当你需要将Compose的状态（如 `mutableStateOf`）转换为Flow，以便使用Flow的操作符（如 `debounce`、`filter`）处理时， `snapshotFlow` 是唯一的工具。它能“监听”状态变化，并将变化后的值发送到Flow中。

1. 实战9：用 `snapshotFlow` 处理状态变化（防抖搜索）

需求：将搜索输入框的状态转换为Flow，用Flow的 `debounce` 操作符实现防抖，和之前的 `LaunchedEffect` 实现异曲同工，但更灵活。

Code block

```

1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.foundation.layout.padding
3  import androidx.compose.material3.OutlinedTextField
4  import androidx.compose.material3.Text
5  import androidx.compose.runtime.*
6  import androidx.compose.ui.Modifier
7  import androidx.compose.ui.unit.dp
8  import kotlinx.coroutines.flow.debounce
9  import kotlinx.coroutines.flow.filter
10
11  @Composable
12  fun SnapshotFlowDemo() {
13      // 搜索输入状态
14      var searchText by remember { mutableStateOf("") }
15      // 搜索结果状态
16      var searchResult by remember { mutableStateOf("") }

```



```
17
18     LaunchedEffect(key1 = Unit) {
19         // 1. 将Compose状态转换为Flow
20         snapshotFlow { searchText }
21         // 2. Flow操作符：过滤空输入
22         .filter { it.isNotBlank() }
23         // 3. Flow操作符：防抖500ms
24         .debounce(500)
25         // 4. 收集Flow数据
26         .collect { text ->
27             // 模拟搜索请求
28             searchResult = "搜索 '$text' 的结果：找到8条数据"
29         }
30     }
31
32     Column(modifier = Modifier.padding(16.dp)) {
33         OutlinedTextField(
34             value = searchText,
35             onChange = { searchText = it },
36             label = { Text("请输入搜索关键词") },
37             modifier = Modifier.fillMaxWidth()
38         )
39         Text(
40             text = searchResult,
41             modifier = Modifier.padding(top = 16.dp)
42         )
43     }
44 }
45
```



核心优势： `snapshotFlow` 让Compose状态可以使用Flow的强大生态，比如 `combine` 合并多个状态、 `distinctUntilChanged` 去重等，处理复杂状态逻辑更灵活。

十、小节回顾：效应API速查表与选择指南

1. 核心API功能速查表

API名称	核心功能	关键特性	适用场景

LaunchedEffect	组件作用域内启动协程	key控制重启，自动取消	页面加载时请求数
rememberCoroutineScope	获取手动控制的协程作用域	手动启动/取消，与组件绑定	按钮点击触发请求消
rememberUpdateState	保鲜状态，不触发效应重启	状态更新不影响效应执行	定时器中使用最新
DisposableEffect	执行需清理的效应	强制写清理逻辑，防止泄漏	注册/取消监听器、源
SideEffect	每次重组后执行轻量效应	无key，每次重组都执行	同步状态到全局变
produceState	从外部数据源生产状态	自动管理数据源生命周期	将Flow、回调转抄Compose状态
derivedStateOf	由其他状态派生新状态	缓存计算结果，依赖变化才更新	根据滚动状态控制隐藏
snapshotFlow	将Compose状态转换为Flow	监听状态变化，支持Flow操作符	用Flow处理状态并)

2. API选择流程图（小白版）

遇到效应需求时，按以下步骤选择API：

- 1. 是否需要启动协程？
是 → 下一步；
- 2. 否 → 看是否需要清理？ 是→DisposableEffect，否→SideEffect；
- 3. 协程是自动启动还是手动启动？
自动（组件加载时）→ LaunchedEffect；
- 4. 手动（按钮点击）→ rememberCoroutineScope；
- 5. 是否需要处理外部数据源/状态转换？
外部数据源→Compose状态 → produceState；
- 6. Compose状态→Flow → snapshotFlow；
- 7. Compose状态→Compose状态（派生）→ derivedStateOf；

3. 开发避坑指南

- 1. 效应不写在Compose函数顶层：所有效应API必须在Compose函数或其他效应内部使用，不能直接写在顶层；
- 2. 协程效应用对作用域：网络请求、延迟等耗时操作必须在协程作用域内（LaunchedEffect/rememberCoroutineScope）执行，不能在UI线程；

3. DisposableEffect必写清理逻辑：注册监听器、资源初始化后，一定要在 `onDispose` 中释放，避免内存泄漏；
4. key参数不滥用：LaunchedEffect/DisposableEffect的key不要传无关状态，否则会导致效应频繁重启；
5. SideEffect不做耗时操作：它运行在UI线程，耗时操作会导致界面卡顿。

十一、课后作业：动手实现“效应综合Demo”

需求：基于本节课知识点，实现一个“新闻列表”页面，包含以下功能，综合运用各类效应API：

1. 页面加载时（组件组合），用 `LaunchedEffect` 发起网络请求（模拟延迟2秒），加载新闻列表数据；
2. 提供“刷新”按钮，用 `rememberCoroutineScope` 手动启动刷新协程，支持“取消刷新”；
3. 列表滚动时，用 `derivedStateOf` 判断是否显示“回到顶部”按钮（滚动超过3项时显示）；
4. 用 `DisposableEffect` 监听Activity生命周期，页面进入后台时停止刷新，回到前台时提示“已恢复”；
5. 用 `snapshotFlow` 将“当前选中的新闻分类”状态转换为Flow，防抖300ms后加载对应分类的新闻；
6. 状态处理：加载中显示进度条，请求失败显示重试按钮，空数据显示“暂无新闻”。

提示：重点练习“API选择逻辑”和“效应生命周期管理”，确保每个效应都“该启动时启动，该停止时停止”，无内存泄漏和重复请求问题。

（注：文档部分内容可能由 AI 生成）